Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №5, часть 1
по дисциплине
«Современные системы программирования»

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды 5
2.4 Задание 1. Запуск и анализ *Producer-Consumer* 5
2.5 Задание 2. Влияние размера буфера 7
2.6 Задание 3. Дополнительные производитель и потребитель 8
2.7 Усложненное задание. Буфер объектов *DataItem* 10
2.8 Задание 4. Диагностика *deadlock* 11
2.9 Задание 5. Исправление *deadlock* 12
2.10 Задание 6. Обнаружение *deadlock* по тайм-ауту 13
2.11 Задание 7. Именованный *Mutex* между процессами 13
3 Ответы на контрольные вопросы 15
Выводы 17
Приложение А (обязательное) Листинги программного кода 18

1 ЦЕЛЬ РАБОТЫ

Изучить принципы параллельного программирования, освоить паттерн «производитель – потребитель» (*Producer-Consumer*), научиться использовать примитивы синхронизации *Mutex* и *Semaphore*, а также освоить методы обнаружения и диагностики взаимоблокировок (*deadlock*): получить стеки зависших потоков, устранить *deadlock* единым порядком захвата блокировок и обнаружить его по тайм-ауту.

2 ХОД РАБОТЫ

Работа выполнена в консольном приложении *C#* для платформы *.NET* 10 (*SDK* 10.0.401). Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для изучения многопоточного программирования в рамках работы требуется выполнить следующие задачи:

1 запустить программу *Producer-Consumer* с семафорами и проанализировать синхронизацию потоков;

2 изменить размер буфера на 2 и на 10 и оценить влияние;

3 добавить еще одного производителя и еще одного потребителя;

4 реализовать хранение в буфере объектов с полями вместо чисел;

5 запустить программу с *deadlock* и выполнить диагностику;

6 исправить *deadlock* одинаковым порядком захвата блокировок;

7 обнаружить *deadlock* с помощью тайм-аута *Monitor.TryEnter*;

8 проверить именованный *Mutex* двумя экземплярами программы.

2.2 Теоретические сведения

Producer-Consumer – классическая задача синхронизации: потоки-производители создают данные и помещают их в общий ограниченный буфер, а потоки-потребители забирают данные из буфера и обрабатывают их. Если буфер заполнен, производитель должен ждать свободного места, если пуст – потребитель должен ждать появления данных.

Для этого используются три примитива. Семафор *empty* (*SemaphoreSlim*) считает свободные места в буфере. Семафор *full* считает занятые места. Мьютекс (*Mutex*) обеспечивает взаимное исключение: в любой момент буфером пользуется только один поток. Семафор со счетчиком пропускает не более заданного числа потоков, а мьютекс пропускает ровно один и помнит своего владельца.

Взаимоблокировка (*deadlock*) – ситуация, в которой два или более потока ждут освобождения ресурсов, занятых друг другом, поэтому ни один не может продолжить работу. Типичная причина – захват нескольких блокировок в разном порядке. Способы борьбы: одинаковый порядок захвата, захват с тайм-аутом (*Monitor.TryEnter*), минимизация числа одновременно удерживаемых блокировок.

Именованный мьютекс (*new Mutex*(*false*, "*Global\\MyAppMutex*")) виден всем процессам операционной системы, поэтому с его помощью можно ограничить запуск приложения одним экземпляром.

2.3 Подготовка среды

Приложение *Lab5* создано как консольный проект *.NET* с целевой платформой *net10.0* (*SDK* 10.0.401). Все задания собраны в одном проекте, режим выбирается аргументом командной строки: *pc*, *pc2*, *pc10*, *pc-more*, *pc-items*, *deadlock*, *deadlock-fixed*, *deadlock-timeout*, *mutex*. Компьютер: процессор *Intel Core i5-1035G1* (4 ядра, 8 логических процессоров).

В коде методических указаний есть дефект: после установки флага *_isRunning = false* производители завершаются, а потребители остаются ожидать *_filledSlots.WaitAsync*() навсегда, и *Task.WaitAll* никогда не возвращает управление. Поэтому ожидание семафоров выполняется с тайм-аутом: *await _emptySlots.WaitAsync*(500) и *await _filledSlots.WaitAsync*(500), а при неудаче цикл проверяет флаг заново. Остальной код не изменен. Также размер буфера и число потоков вынесены в параметры метода *Run*, чтобы выполнить задания 2 и 3 без правки кода.

2.4 Задание 1. Запуск и анализ *Producer-Consumer*

Два производителя и три потребителя работают с общим буфером на 5 элементов в течение 5 секунд. Код производителя и потребителя приведен в приложении А.

Результат запуска показан на рисунках 2.1 и 2.2.

```
> dotnet run -c Release -- pc
=== Producer-Consumer with Semaphore (буфер 5, производителей 2, потребителей 3) ===

[Producer 1] Добавлен элемент: 25. В буфере: 1 элементов.
[Producer 2] Добавлен элемент: 77. В буфере: 2 элементов.
[Consumer 3] Извлечен элемент: 25. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 77. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 47. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 47. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 17. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 17. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 66. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 66. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 36. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 36. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 80. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 80. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 23. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 23. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 76. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 76. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 1. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 1. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 3. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 3. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 32. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 32. В буфере: 0 элементов.
```

Рисунок 2.1 – Работа *Producer-Consumer*, буфер 5 (начало)

Производители и потребители чередуются: каждый добавленный элемент почти сразу извлекается, значение счетчика «В буфере» остается в пределах 0–2.

Окончание вывода показано на рисунке 2.2.

```
[Producer 2] Добавлен элемент: 38. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 38. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 70. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 70. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 68. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 68. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 73. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 73. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 9. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 9. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 28. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 28. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 33. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 33. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 70. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 70. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 89. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 89. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 95. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 95. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 16. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 16. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 65. В буфере: 1 элементов.
[Consumer 1] Завершен.
[Producer 1] Завершен.
[Consumer 2] Завершен.
[Consumer 3] Завершен.
[Producer 2] Завершен.

Все потоки завершены.
```

Рисунок 2.2 – Работа *Producer-Consumer*, буфер 5 (окончание)

За 5 секунд добавлено 26 элементов, извлечено 25, один остался в буфере. После установки флага остановки все пять задач вывели сообщение «Завершен».

Синхронизация потоков устроена так. Производитель сначала занимает место через *_emptySlots.WaitAsync*(), затем на время записи в очередь захватывает *_mutex*, после записи освобождает мьютекс и увеличивает счетчик занятых мест *_filledSlots.Release*(). Потребитель действует симметрично: ждет *_filledSlots*, извлекает элемент под мьютексом и освобождает место *_emptySlots.Release*(). Благодаря этому записи не пропадают и порядок «элемент сначала добавлен, потом извлечен» не нарушается.

2.5 Задание 2. Влияние размера буфера

Размер буфера изменен на 2 и на 10. Результаты показаны на рисунках 2.3 и 2.4.

```
> dotnet run -c Release -- pc2
=== Producer-Consumer with Semaphore (буфер 2, производителей 2, потребителей 3) ===

[Producer 2] Добавлен элемент: 77. В буфере: 1 элементов.
[Producer 1] Добавлен элемент: 25. В буфере: 2 элементов.
[Consumer 3] Извлечен элемент: 77. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 25. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 47. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 47. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 17. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 17. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 66. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 66. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 36. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 36. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 80. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 80. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 23. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 23. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 76. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 76. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 1. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 1. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 3. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 3. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 32. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 32. В буфере: 0 элементов.
```

Рисунок 2.3 – Работа программы при размере буфера 2

При размере 2 за 5 секунд добавлено 26 элементов и извлечено 25, наибольшее число элементов в буфере равно 2.

Тема «работа программы при размере буфера 10» показана на рисунке 2.4.

```
> dotnet run -c Release -- pc10
=== Producer-Consumer with Semaphore (буфер 10, производителей 2, потребителей 3) ===

[Producer 1] Добавлен элемент: 25. В буфере: 1 элементов.
[Producer 2] Добавлен элемент: 77. В буфере: 2 элементов.
[Consumer 1] Извлечен элемент: 25. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 77. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 47. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 47. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 17. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 17. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 66. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 66. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 36. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 36. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 80. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 80. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 23. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 23. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 76. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 76. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 1. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 1. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 3. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 3. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 32. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 32. В буфере: 0 элементов.
```

Рисунок 2.4 – Работа программы при размере буфера 10

При размере 10 результат такой же: добавлено 26 элементов, извлечено 25, наибольшее число элементов в буфере равно 2.

Поведение программы не изменилось, потому что буфер ни разу не заполнился. Два производителя создают элемент раз в 200–600 мс, а три потребителя обрабатывают элемент за 300–800 мс, то есть суммарно забирают данные быстрее, чем производители их создают, и очередь почти всегда пуста. Размер буфера влияет на работу только тогда, когда производители быстрее потребителей: при малом буфере производитель блокируется на *_emptySlots* и общая скорость определяется потребителями, при большом буфере накапливается больший запас элементов, производитель дольше не ждет, но расходуется больше памяти.

2.6 Задание 3. Дополнительные производитель и потребитель

Добавлены третий производитель и четвертый потребитель (режим *pc-more*). Результат показан на рисунках 2.5 и 2.6.

```
> dotnet run -c Release -- pc-more
=== Producer-Consumer with Semaphore (буфер 5, производителей 3, потребителей 4) ===

[Producer 1] Добавлен элемент: 25. В буфере: 1 элементов.
[Producer 3] Добавлен элемент: 30. В буфере: 2 элементов.
[Producer 2] Добавлен элемент: 77. В буфере: 3 элементов.
[Consumer 1] Извлечен элемент: 25. В буфере: 2 элементов.
[Consumer 2] Извлечен элемент: 30. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 77. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 47. В буфере: 1 элементов.
[Consumer 4] Извлечен элемент: 47. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 17. В буфере: 1 элементов.
[Producer 3] Добавлен элемент: 86. В буфере: 2 элементов.
[Consumer 1] Извлечен элемент: 17. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 86. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 66. В буфере: 1 элементов.
[Consumer 4] Извлечен элемент: 66. В буфере: 0 элементов.
[Producer 3] Добавлен элемент: 56. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 56. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 11. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 11. В буфере: 0 элементов.
[Producer 3] Добавлен элемент: 25. В буфере: 1 элементов.
[Producer 1] Добавлен элемент: 36. В буфере: 2 элементов.
[Consumer 3] Извлечен элемент: 25. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 36. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 80. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 80. В буфере: 0 элементов.
[Producer 3] Добавлен элемент: 35. В буфере: 1 элементов.
[Consumer 4] Извлечен элемент: 35. В буфере: 0 элементов.
```

Рисунок 2.5 – Три производителя и четыре потребителя (начало)

Новые потоки сразу включились в работу: в журнале появились записи *Producer* 3 и *Consumer* 4.

Продолжение вывода показано на рисунке 2.6.

```
[Producer 2] Добавлен элемент: 23. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 23. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 11. В буфере: 1 элементов.
[Producer 2] Добавлен элемент: 76. В буфере: 2 элементов.
[Consumer 3] Извлечен элемент: 11. В буфере: 1 элементов.
[Producer 3] Добавлен элемент: 50. В буфере: 2 элементов.
[Consumer 2] Извлечен элемент: 76. В буфере: 1 элементов.
[Producer 2] Добавлен элемент: 1. В буфере: 2 элементов.
[Producer 1] Добавлен элемент: 3. В буфере: 3 элементов.
[Consumer 4] Извлечен элемент: 50. В буфере: 2 элементов.
[Consumer 1] Извлечен элемент: 1. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 3. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 32. В буфере: 1 элементов.
[Producer 3] Добавлен элемент: 69. В буфере: 2 элементов.
[Producer 2] Добавлен элемент: 38. В буфере: 3 элементов.
[Consumer 2] Извлечен элемент: 32. В буфере: 2 элементов.
[Consumer 1] Извлечен элемент: 69. В буфере: 1 элементов.
[Consumer 3] Извлечен элемент: 38. В буфере: 0 элементов.
[Producer 3] Добавлен элемент: 9. В буфере: 1 элементов.
[Consumer 4] Извлечен элемент: 9. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 70. В буфере: 1 элементов.
[Consumer 2] Извлечен элемент: 70. В буфере: 0 элементов.
[Producer 1] Добавлен элемент: 68. В буфере: 1 элементов.
[Consumer 1] Извлечен элемент: 68. В буфере: 0 элементов.
[Producer 2] Добавлен элемент: 73. В буфере: 1 элементов.
[Producer 3] Добавлен элемент: 13. В буфере: 2 элементов.
[Consumer 4] Извлечен элемент: 73. В буфере: 1 элементов.
[Producer 2] Добавлен элемент: 9. В буфере: 2 элементов.
[Consumer 2] Извлечен элемент: 13. В буфере: 1 элементов.
```

Рисунок 2.6 – Три производителя и четыре потребителя (продолжение)

Всего добавлено 39 элементов, извлечено 36, наибольшее число элементов в буфере равно 4, все семь задач завершились. Каждый элемент извлечен ровно один раз, значения счетчика «В буфере» не выходят за пределы от 0 до размера буфера, поэтому программа работает корректно.

2.7 Усложненное задание. Буфер объектов *DataItem*

Вместо чисел в буфере хранятся объекты класса *DataItem* с полями *Id* и *Payload*. Идентификатор выдается через *Interlocked.Increment*, чтобы два производителя не создали одинаковые номера. Код приведен в приложении А.

Результат показан на рисунках 2.7 и 2.8.

```
> dotnet run -c Release -- pc-items
=== Producer-Consumer: буфер хранит объекты DataItem ===

[Producer 2] Добавлен DataItem { Id = 1, Payload = "данные от производителя 2" }. В буфере: 1.
[Producer 1] Добавлен DataItem { Id = 2, Payload = "данные от производителя 1" }. В буфере: 2.
[Consumer 1] Извлечен DataItem { Id = 1, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 2, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 3, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 3, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 4, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 4, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 5, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 5, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 6, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 6, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 7, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 7, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 8, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 8, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 9, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 9, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 10, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 10, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 11, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 11, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 12, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 12, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 13, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 13, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 14, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 14, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 15, Payload = "данные от производителя 1" }. В буфере: 1.
```

Рисунок 2.7 – Буфер объектов *DataItem* (начало)

В журнале видны номера и тексты объектов, извлекаются они в том же порядке, в котором были добавлены (очередь работает по принципу «первым пришел – первым вышел»).

Окончание вывода показано на рисунке 2.8.

```
[Consumer 2] Извлечен DataItem { Id = 15, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 16, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 16, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 17, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 17, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 18, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 1] Извлечен DataItem { Id = 18, Payload = "данные от производителя 2" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 19, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 19, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 20, Payload = "данные от производителя 2" }. В буфере: 1.
[Producer 2] Добавлен DataItem { Id = 21, Payload = "данные от производителя 2" }. В буфере: 2.
[Consumer 2] Извлечен DataItem { Id = 20, Payload = "данные от производителя 2" }. В буфере: 1.
[Producer 1] Добавлен DataItem { Id = 22, Payload = "данные от производителя 1" }. В буфере: 2.
[Consumer 1] Извлечен DataItem { Id = 21, Payload = "данные от производителя 2" }. В буфере: 1.
[Consumer 3] Извлечен DataItem { Id = 22, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 1] Добавлен DataItem { Id = 23, Payload = "данные от производителя 1" }. В буфере: 1.
[Consumer 2] Извлечен DataItem { Id = 23, Payload = "данные от производителя 1" }. В буфере: 0.
[Producer 2] Добавлен DataItem { Id = 24, Payload = "данные от производителя 2" }. В буфере: 1.
[Producer 1] Добавлен DataItem { Id = 25, Payload = "данные от производителя 1" }. В буфере: 2.
[Producer 2] Добавлен DataItem { Id = 26, Payload = "данные от производителя 2" }. В буфере: 3.
[Consumer 3] Извлечен DataItem { Id = 24, Payload = "данные от производителя 2" }. В буфере: 2.
[Producer 2] Добавлен DataItem { Id = 27, Payload = "данные от производителя 2" }. В буфере: 3.
[Consumer 1] Извлечен DataItem { Id = 25, Payload = "данные от производителя 1" }. В буфере: 2.
[Consumer 2] Извлечен DataItem { Id = 26, Payload = "данные от производителя 2" }. В буфере: 1.
[Producer 1] Добавлен DataItem { Id = 28, Payload = "данные от производителя 1" }. В буфере: 2.
[Producer 2] Завершен.
[Consumer 1] Завершен.
[Consumer 2] Завершен.
[Producer 1] Завершен.
[Consumer 3] Завершен.

Все потоки завершены.
```

Рисунок 2.8 – Буфер объектов *DataItem* (окончание)

Добавлено 28 объектов, извлечено 26, два остались в буфере. Все задачи завершены.

2.8 Задание 4. Диагностика *deadlock*

Программа с искусственным *deadlock* запускает две задачи: *Thread1* захватывает *_lockA*, затем после паузы *_lockB*, а *Thread2* – сначала *_lockB*, затем *_lockA*. Код приведен в приложении А.

Результат запуска показан на рисунке 2.9.

```
> dotnet run -c Release -- deadlock
=== Демонстрация Deadlock ===

Ожидание завершения потоков... (программа зависнет)
Для диагностики откройте окно 'Parallel Stacks' в VS.
Thread2: захватил lockB
Thread1: захватил lockA

[через 5 секунд программа не завершилась – deadlock]
```

Рисунок 2.9 – Зависание программы с *deadlock*

Каждый поток успел захватить свою блокировку и вывести сообщение, после этого вывод остановился. Через 5 секунд программа не завершилась.

Диагностика выполнена по остановленному процессу. Окна *Visual Studio Parallel Stacks* и *Call Stack* показывают стеки потоков, поэтому для снимка использован консольный аналог – утилита *dotnet-stack*, которая выводит стеки всех потоков запущенного процесса. Результат показан на рисунке 2.10.

```
> dotnet-stack report -p <PID процесса Lab5>
Thread (0x4EE8):
  System.Threading.Monitor.Wait()
  System.Threading.ManualResetEventSlim.Wait()
  System.Threading.Tasks.Task.WaitAllBlockingCore()
  System.Threading.Tasks.Task.WaitAllCore()
  System.Threading.Tasks.Task.WaitAll()
  Lab5!Lab5.DeadlockDemo.Run()
  Lab5!Program.<Main>$()

Thread (0x4F20):
  System.Threading.Monitor.Enter_Slowpath()
  System.Threading.Monitor.Enter()
  Lab5!Lab5.DeadlockDemo.Thread1()
  Lab5!Lab5.DeadlockDemo+<>c.<Run>b__2_0()

Thread (0x5E4):
  System.Threading.Monitor.Enter_Slowpath()
  System.Threading.Monitor.Enter()
  Lab5!Lab5.DeadlockDemo.Thread2()
  Lab5!Lab5.DeadlockDemo+<>c.<Run>b__2_1()
```

Рисунок 2.10 – Стеки потоков зависшей программы

Основной поток стоит в *Task.WaitAll*, ожидая обе задачи. Потоки *Thread1* и *Thread2* оба находятся в *Monitor.Enter_Slowpath*, то есть ждут блокировку, которую удерживает другой поток. Два потока, каждый из которых ждет ресурс второго, образуют цикл ожидания – это и есть *deadlock*.

2.9 Задание 5. Исправление *deadlock*

Для исправления поток *Thread2* изменен так, чтобы захватывать блокировки в том же порядке, что и *Thread1*: сначала *_lockA*, потом *_lockB*. Код приведен в приложении А.

Результат показан на рисунке 2.11.

```
> dotnet run -c Release -- deadlock-fixed
=== Deadlock исправлен: одинаковый порядок захвата ===

Thread2: захватил lockA
Thread2: захватил lockB
Thread1: захватил lockA
Thread1: захватил lockB
Программа завершена
```

Рисунок 2.11 – Исправленная программа

Потоки захватывают блокировки поочередно, сначала один поток проходит обе блокировки, затем второй. Программа завершилась с сообщением «Программа завершена».

2.10 Задание 6. Обнаружение *deadlock* по тайм-ауту

Операторы *lock* заменены на *Monitor.TryEnter*(объект, *TimeSpan*): если блокировку не удалось получить за 3 секунды, поток выводит сообщение. Порядок захвата оставлен разным, чтобы *deadlock* возник. Код приведен в приложении А.

Результат показан на рисунке 2.12.

```
> dotnet run -c Release -- deadlock-timeout
=== Обнаружение deadlock по тайм-ауту (Monitor.TryEnter) ===

Thread1: захватил lockA
Thread2: захватил lockB
Thread1: не удалось захватить lockB за 3 секунды – возможен deadlock
Thread2: не удалось захватить lockA за 3 секунды – возможен deadlock
Программа завершена: зависания нет, потоки сообщили о тайм-ауте
```

Рисунок 2.12 – Обнаружение *deadlock* по тайм-ауту

Оба потока захватили свои блокировки, не смогли получить вторые за 3 секунды и сообщили о возможном *deadlock*. Программа при этом не зависла, а завершилась.

2.11 Задание 7. Именованный *Mutex* между процессами

Программа создает именованный мьютекс *Global\\MyAppMutex* и пытается захватить его в течение 3 секунд. Код приведен в приложении А.

Запущены два экземпляра программы. Первый экземпляр показан на рисунке 2.13.

```
> dotnet run -c Release -- mutex   (экземпляр 1)
Попытка захвата глобального мьютекса...
Мьютекс захвачен. Программа выполняется.
Нажмите Enter для освобождения мьютекса...
Мьютекс освобожден.
```

Рисунок 2.13 – Первый экземпляр программы

Первый экземпляр захватил мьютекс и ждет нажатия *Enter*. После нажатия мьютекс освобожден.

Второй экземпляр, запущенный в это время, показан на рисунке 2.14.

```
> dotnet run -c Release -- mutex   (экземпляр 2)
Попытка захвата глобального мьютекса...
Не удалось захватить мьютекс (возможно, уже запущен другой экземпляр).
```

Рисунок 2.14 – Второй экземпляр программы

Второй экземпляр за 3 секунды не получил мьютекс и вывел сообщение о невозможности захвата, как требует задание.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чем разница между *Mutex* и *Semaphore*?

Mutex пропускает в защищенную область ровно один поток и знает своего владельца: освободить мьютекс может только тот поток, который его захватил. *Semaphore* хранит счетчик и пропускает до заданного числа потоков одновременно, владельца у него нет, освободить его может любой поток. Мьютекс может быть именованным и работать между процессами.

2. Что такое семафор со счетчиком и для чего он используется в *Producer-Consumer*?

Семафор со счетчиком содержит число доступных «разрешений». Операция ожидания уменьшает счетчик, а если он равен нулю – блокирует поток. Операция *Release* увеличивает счетчик и пробуждает ожидающего. В *Producer-Consumer* один семафор считает свободные места (производитель ждет, когда буфер заполнен), другой – занятые места (потребитель ждет, когда буфер пуст).

3. Какие способы диагностики *deadlock* существуют в *Visual Studio*?

После остановки зависшей программы командой *Break All* используют окна *Parallel Stacks* (граф стеков всех потоков и цикл ожидания), *Threads* (состояние каждого потока, зависшие потоки в состоянии ожидания), *Call Stack* (место, где поток остановился) и *Parallel Watch*. Дополнительно *deadlock* обнаруживают по тайм-аутам блокировок и по дампам процесса.

4. Чем отличается *lock* от *Mutex*?

lock – конструкция языка *C#*, использующая *Monitor* на объекте. Она быстрее, работает только внутри одного процесса и автоматически освобождает блокировку при выходе из блока. *Mutex* – объект операционной системы, работает медленнее, может быть именованным и синхронизировать разные процессы, но требует явного вызова *ReleaseMutex*.

5. Как избежать *deadlock*'ов при проектировании многопоточных приложений?

Захватывать блокировки всегда в одном и том же порядке, удерживать их как можно меньше и как можно короче, не вызывать чужой код под блокировкой, использовать захват с тайм-аутом (*Monitor.TryEnter*) и потокобезопасные коллекции вместо ручных блокировок.

6. Что такое "голодание" (*starvation*) потоков и как оно отличается от *deadlock*?

Голодание – ситуация, в которой поток долго или бесконечно не получает ресурс или процессорное время, потому что его постоянно опережают другие потоки (например, с более высоким приоритетом). При этом система в целом продолжает работать. При *deadlock* потоки блокируют

друг друга и не могут продолжить вообще, пока ситуация не будет разрешена извне.

ВЫВОДЫ

Изучен паттерн *Producer-Consumer*: реализована синхронизация потоков двумя семафорами и мьютексом, проверена работа при размерах буфера 2, 5 и 10, при трех производителях и четырех потребителях и при хранении в буфере объектов *DataItem*. Показано, что размер буфера влияет на работу только при производителях, опережающих потребителей. Создан и продиагностирован *deadlock* по стекам потоков, он исправлен единым порядком захвата блокировок и обнаружен по тайм-ауту *Monitor.TryEnter*. С помощью именованного *Mutex* второй экземпляр программы не смог запуститься одновременно с первым. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги программного кода

Листинг А.1 – Производитель и потребитель (задание 1)

```
// Метод производителя
static async Task Producer(int id)
{
    Random rand = new Random(id);
    while (_isRunning)
    {
        // Ждем, пока освободится место в буфере (с тайм-аутом, чтобы после остановки
поток мог завершиться)
        if (!await _emptySlots.WaitAsync(500)) continue;
        // Захватываем мьютекс для безопасной работы с буфером
        _mutex.WaitOne();
        try
        {
            int item = rand.Next(1, 100);
            _buffer.Enqueue(item);
            Console.WriteLine($"[Producer {id}] Добавлен элемент: {item}. В буфере:
{_buffer.Count} элементов.");
        }
        finally
        {
            _mutex.ReleaseMutex();
        }
        // Увеличиваем счетчик занятых мест
        _filledSlots.Release();
        // Имитация времени на производство
        await Task.Delay(rand.Next(200, 600));
    }
    Console.WriteLine($"[Producer {id}] Завершен.");
}

// Метод потребителя
static async Task Consumer(int id)
{
    Random rand = new Random(id + 10);
    while (_isRunning)
    {
        // Ждем, пока появится хотя бы один элемент в буфере (с тайм-аутом по той же
причине)
        if (!await _filledSlots.WaitAsync(500)) continue;
        // Захватываем мьютекс
        _mutex.WaitOne();
        int item;
        try
        {
            item = _buffer.Dequeue();
```

```
            Console.WriteLine($"[Consumer {id}] Извлечен элемент: {item}. В буфере:
{_buffer.Count} элементов.");
        }
        finally
        {
            _mutex.ReleaseMutex();
        }
        // Увеличиваем счетчик свободных мест
        _emptySlots.Release();
        // Имитация обработки данных
        await Task.Delay(rand.Next(300, 800));
    }
    Console.WriteLine($"[Consumer {id}] Завершен.");
}
```

Листинг А.2 – Класс *DataItem* и производитель объектов

```
// Усложненное задание: в буфере хранятся объекты с полями
public class DataItem
{
    public int Id;
    public string Payload = "";
}

var item = new DataItem { Id = Interlocked.Increment(ref _nextId), Payload = $"данные от
производителя {id}" };
_buffer.Enqueue(item);
Console.WriteLine($"[Producer {id}] Добавлен DataItem {{ Id = {item.Id}, Payload =
\"{item.Payload}\" }}. В буфере: {_buffer.Count}.");
```

Листинг А.3 – Потоки с разным порядком захвата блокировок

```
static void Thread1()
{
    lock (_lockA)
    {
        Console.WriteLine("Thread1: захватил lockA");
        Thread.Sleep(1000); // Имитация работы
        lock (_lockB)
        {
            Console.WriteLine("Thread1: захватил lockB");
        }
    }
}

static void Thread2()
{
    lock (_lockB)
    {
        Console.WriteLine("Thread2: захватил lockB");
        Thread.Sleep(1000); // Имитация работы
        lock (_lockA)
        {
```

```
            Console.WriteLine("Thread2: захватил lockA");
        }
    }
}
```

Листинг А.4 – Одинаковый порядок захвата блокировок

```
static void Thread2Fixed()
{
    lock (_lockA)
    {
        Console.WriteLine("Thread2: захватил lockA");
        Thread.Sleep(1000); // Имитация работы
        lock (_lockB)
        {
            Console.WriteLine("Thread2: захватил lockB");
        }
    }
}
```

Листинг А.5 – Захват блокировок с тайм-аутом

```
static void TakeBoth(string name, object first, string firstName, object second, string
secondName)
{
    if (!Monitor.TryEnter(first, TimeSpan.FromSeconds(3)))
    {
        Console.WriteLine($"{name}: не удалось захватить {firstName} за 3 секунды");
        return;
    }
    try
    {
        Console.WriteLine($"{name}: захватил {firstName}");
        Thread.Sleep(1000); // Имитация работы
        if (Monitor.TryEnter(second, TimeSpan.FromSeconds(3)))
        {
            try { Console.WriteLine($"{name}: захватил {secondName}"); }
            finally { Monitor.Exit(second); }
        }
        else
        {
            Console.WriteLine($"{name}: не удалось захватить {secondName} за 3 секунды –
возможен deadlock");
        }
    }
    finally
    {
        Monitor.Exit(first);
    }
}
```

Листинг А.6 – Именованный *Mutex*

```
public static void Run()
{
    using (Mutex mutex = new Mutex(false, "Global\\MyAppMutex"))
    {
        Console.WriteLine("Попытка захвата глобального мьютекса...");
        if (mutex.WaitOne(TimeSpan.FromSeconds(3)))
        {
            try
            {
                Console.WriteLine("Мьютекс захвачен. Программа выполняется.");
                Console.WriteLine("Нажмите Enter для освобождения мьютекса...");
                Console.ReadLine();
            }
            finally
            {
                mutex.ReleaseMutex();
                Console.WriteLine("Мьютекс освобожден.");
            }
        }
        else
        {
            Console.WriteLine("Не удалось захватить мьютекс (возможно, уже запущен
другой экземпляр).");
        }
    }
}
```